OneLocal Rider (OnLatur) — End-to-end V1 specification

Version: V1 · **Date:** 2026-09-26 · **Status:** decided (product owner by delegation) · **Supersedes:** screens and sign-in of `PRD_ONLATUR_RIDER_V1.md` (its dispatch, cash and alert rules still apply) **Design source:** Figma "rider app" (`RWnNje0um6Ugp9TC7kNmnU` , Frame 1 and `rider-state-machine`), read screen by screen on 2026-09-26. **Builds on:** `PLATFORM_FOUNDATIONS.md` (D1–D15), the One Local server (`frontend/app/api/*`), the partner API already built (`/api/partner/*`), and the delivery-area model (`ol_service_areas`). **Names:** product/domain **OnLatur** (delivery and riders); app shown to riders as **OneLocal Rider** (the design's brand, one brand family with the customer app).

0. Decisions at a glance

#	Decision	Why / what lost
R1	**V1 rider types: Store Rider and Solo Rider.** Taxi Partner and vehicle rental stay in the design and show "Coming soon" (the design already marks Taxi as coming soon)	Taxi carries passengers — a different licence, insurance and product. Rental needs partner contracts (Zypp, Yulu)
R2	**Sign-in is phone number + OTP**, as designed. Supabase phone auth with an Indian SMS provider (MSG91) and DLT registration; until then, Supabase test numbers for the pilot riders	Google sign-in (the earlier PRD) lost: riders are identified by phone everywhere in India, and the design is built around it
R3	**Geography is State → City → Zone → Hub**, stored as map shapes (PostGIS), not text. Riders pick a zone and a start hub; orders are dispatched inside a zone and its neighbours	A centre-plus-radius per city (today's `ol_service_areas`) can't express "Latur East" or keep a rider near their hub
R4	**Every place is a coordinate first, text second.** Pickup and drop carry latitude/longitude, the accuracy and who set it; navigation opens Google Maps on the coordinate, never on the address text	Text-address navigation is the main cause of wrong-address trips
R5	**The 12-state job engine from the design** is the single source of truth for a delivery; every state has exactly one exception path	The earlier 9-state list lost detail the design needs (pickup verified, handover, proof)
R6	**Dispatch: linked-store riders first, then nearby Solo riders**, ranked by road ETA; offers last 30 s in Manual mode; Auto-Accept assigns instantly (design: Rider Priority Rules, Acceptance Mode)	—
R7	**Proof: OTP by default**; photo when the customer can't be reached or asked for contactless; signature for Pick & Drop parcels	One method for everything lost: OTP is strongest, but contactless food and parcels need other proofs
R8	**Live location only during a job** (plus a 30 s heartbeat while online); precise GPS every 5–10 s while moving on a job, never when offline	Always-on tracking lost: battery, privacy and Play Store background-location review
R9	**The rider app is a second Expo app in this repository** (`OL-ui/rider/`), calling the same One Local server under `/api/rider/*`	A separate backend lost: orders, customers and stores already live behind this API
R10	**Earnings rules are configuration** (base, distance, peak, tip pass-through) set in the Master per city; the design's numbers are samples	Hard-coded rates lost: they differ by city and change weekly
R11	**Payouts: bank and UPI details are collected and verified in V1; money is paid out manually (weekly) until a payout provider is chosen**; "Withdraw earnings" shows coming soon	Instant withdrawal needs RazorpayX/Cashfree Payouts and a current account
R12	**Combined / multi-drop delivery is V2** (the design labels it "hidden in V1 platform build")	—

1. Screen inventory (from Figma) and V1 status

Each Figma frame name becomes an app route. ✅ = V1, 🕓 = shown as "coming soon", ⏭ = V2.

1.1 First launch and sign-in

#	Figma frame	Route	What it does	V1
1	Splash (OneLocal wordmark)	`/`	Brand splash while session and config load	✅
2–4	Intro: *Work on Your Own Terms · Transparent Daily Earnings · Simple Accept & Go Flow*	`/intro`	3-card carousel, Skip, Next, Get Started	✅
5	*Find Orders Nearby*	`/permissions/location`	Location permission (while using app) — Allow / Not now	✅
6	*Welcome back*	`/sign-in`	Phone number, Continue, "New Rider? Register here"	✅
7	*Apply as Rider*	`/register`	Full name, phone, referral code (optional), accept Terms & Privacy, Send OTP	✅
8	*Enter OTP*	`/otp`	6-digit OTP, auto-read SMS (Android), resend in 30 s, Verify & Proceed	✅
9	*Rider Profile*	`/onboarding/profile`	Photo, name as in Aadhaar, emergency contact, app language (English, हिंदी, ಕನ್ನಡ → add मराठी for Latur)	✅

1.2 Location, hub and rider type

#	Figma frame	Route	What it does	V1
10	`location-selection` — *Your Location*	`/onboarding/zone`	Map with detected zone; pick operating zone inside the state	✅
11	`address-setup` — *Primary Start Hub*	`/onboarding/hub`	City/region, starting address or hub point, landmark; "orders within 3–5 km of this point first"	✅
12	`notification-permission` — *Instant Gig Alerts*	`/permissions/notifications`	Push permission with sample alert	✅
13	`location-confirmation` — *Is this correct?*	`/onboarding/hub/confirm`	Pin on map, "Yes, confirm hub" / "No, change pin"	✅
14	`rider-type-selection` — *How do you ride?*	`/onboarding/type`	Store Rider · Solo Rider · Taxi Partner (coming soon)	✅ (Taxi 🕓)

1.3 Store Rider path

#	Figma frame	Route	What it does	V1
15	`store-link` — *Link to Store*	`/onboarding/store/link`	Merchant invite code → matched merchant card (name, zone, manager, daily pay)	✅
16	`store-kyc` — *KYC Verification*	`/onboarding/kyc`	Aadhaar & PAN (DigiLocker), selfie liveness, driving licence photo; progress Submitted → Under review → Verified	✅
17	`store-vehicle` — *Your Vehicle*	`/onboarding/vehicle`	2/3/4-wheeler, registration number, RC photo; Finish setup & go online	✅
18	*Delivery Categories* (step 1/5)	`/onboarding/categories`	Quick Drop, On-Order Delivery, Pick & Drop	✅
19	*Which store(s) will you ride for?* (2/5)	`/onboarding/stores`	Available to multiple stores toggle; choose preferred hub stores	✅
20	*Acceptance Mode* (3/5)	`/onboarding/acceptance`	Auto-Accept (recommended) or Manual (30 s window)	✅
21	*Rider Priority Rules* (4/5)	`/onboarding/priority`	Linked store first; nearby Solo fallback	✅ (info)
22	*Approval Pending* (store)	`/status`	Checks done, estimated dispatch time, Refresh status	✅

1.4 Solo Rider path

#	Figma frame	Route	What it does	V1
23	*Rider KYC setup* (3/4)	`/onboarding/kyc`	Same KYC as 16, verified via DigiLocker	✅
24	*Vehicle & Ownership* (4/4)	`/onboarding/vehicle`	"I own a vehicle" / "I need to rent"; type; registration; RC	✅ (rent 🕓)
25	*Rental Marketplace*	`/onboarding/rental`	Zypp EV ₹149/day, Yulu Wynn ₹99/day	⏭
26	*Delivery Categories* (1/4)	`/onboarding/categories`	As 18	✅
27	*Acceptance Mode* (2/4)	`/onboarding/acceptance`	As 20	✅
28	*Select your bank details* (3/4)	`/onboarding/payout/bank`	Account holder, account number, IFSC; encrypted	✅
29	*How should we pay you?*	`/onboarding/payout/upi`	UPI ID with Verify (shows verified name)	✅
30	*Approval Pending* (solo)	`/status`	"Usually under 24 hours; we'll SMS you"	✅

1.5 Taxi Partner path (design complete, not in V1)

`Taxi Vehicle Type` · `Commercial Permit` · `Service Capabilities` · `Payout Gateway` · `Taxi Setup Ready` · `taxi-approval-pending` — ⏭ V2.

1.6 Account status

Figma frame	Route	Meaning	V1
`application-submitted`	`/status`	Registration packet received	✅
`verification-pending`	`/status`	Agents reviewing profile, background, vehicle	✅
`approved` — *You're Approved!*	`/status`	Ready to ride; first-ride bonus note; Go online now	✅
`rejected` — *Application Rejected*	`/status`	Reason shown; Re-upload documents	✅
`documents-expired`	`/status`	Named document expired; upload valid one	✅
`account-suspended`	`/status`	Access restricted; contact support	✅

1.7 Working day

#	Figma frame	Route	What it does	V1
31	`home-offline` — *Good morning!*	`/(tabs)/home`	You are offline; yesterday's earnings; completed jobs; **GO ONLINE**	✅
32	`home-online` — *You're online*	`/(tabs)/home`	Linked store banner; today's earnings; waiting radar; GO OFFLINE	✅
33	`job-offer-manual`	`/offer/[id]`	Estimated payout, surge, store, pickup and drop distance, time, note; countdown; Decline / Accept job	✅
34	`job-auto-accepted`	`/offer/[id]`	"Job auto-accepted! Connecting to merchant dispatch"	✅
35	`current-job`	`/job/[id]`	Map, stepper At store → Delivery, pickup and drop, Start navigation	✅
36	`at-pickup`	`/job/[id]/pickup`	Merchant order id, item count, package checklist, Verify pickup	✅
37	`pickup-verification`	`/job/[id]/pickup/verify`	Scan QR on receipt / enter code / bypass (with reason)	✅
38	`pickup-confirmed`	`/job/[id]/pickup/done`	Verification success, milestone bonus note, Proceed to delivery	✅
39	`order-not-ready`	`/job/[id]/pickup/waiting`	Elapsed wait timer; wait compensation after threshold; report delay; contact merchant	✅
40	`package-mismatch`	`/job/[id]/pickup/mismatch`	Scanned ≠ expected; report wrong barcode	✅
41	`package-incomplete`	`/job/[id]/pickup/incomplete`	Missing item flagged; report incomplete	✅
42	`navigation-handoff`	`/job/[id]/navigate`	Package picked up; destination; distance, ETA; Start navigation (Google Maps)	✅
43	`active-delivery`	`/job/[id]`	On the way; customer card with call; ETA; Arrived at destination	✅

44	`arrival`	`/job/[id]/arrived`	"You have arrived"; drop address; call customer; Start delivery verification	✅
45	`delivery-confirmation`	`/job/[id]/proof`	Choose Secure OTP / Photo / Signature	✅
46	`proof-otp`	`/job/[id]/proof/otp`	Keypad; 4-digit code shared with customer	✅
47	`proof-photo`	`/job/[id]/proof/photo`	Camera, retake, submit	✅
48	`proof-signature`	`/job/[id]/proof/signature`	Sign on screen, clear, submit	✅
49	`delivery-success`	`/job/[id]/done`	Delivered; trip earnings; bonus; order id, customer, time; Back to home	✅

1.8 Problems and safety

Figma frame	Route	What it does	V1
`delivery-failed` — *Report Delivery Issue*	`/job/[id]/issue`	Customer unavailable · wrong address · customer refused · cannot access location · safety issue · other	✅
`customer-unavailable`	`/job/[id]/issue/unavailable`	Mandatory wait timer; call / chat; protocol; escalate unlocks at 0	✅
`wrong-address`	`/job/[id]/issue/address`	Stated address vs rider's GPS (distance shown); share current location; call customer; submit address dispute	✅
`safety-exception`	`/job/[id]/sos`	Trigger emergency SOS; report accident/dispute; call rider helpline	✅
`connectivity-states`	global banner	Online + heartbeat · syncing queued tasks · offline mode · sync error retrying	✅

1.9 Money, history, profile

Figma frame	Route	What it does	V1
`earnings-today`	`/(tabs)/earnings`	Today's payout, deliveries, average per trip, split (order pay, milestone bonus, tips); Withdraw	✅ (Withdraw 🕓)
`earnings-weekly`	`/earnings/week`	Weekly total, daily chart, insights; download PDF	✅
`earnings-breakdown`	`/earnings/job/[id]`	Route and payout calculation (base, distance, peak, tip)	✅
`job-history` — *Task History*	`/(tabs)/tasks`	Today / yesterday list with delivered/failed	✅
`delivery-detail` — *Task #… Detail*	`/tasks/[id]`	Timeline, verification, earnings	✅
`profile` — *Rider Profile*	`/(tabs)/profile`	Rider type, vehicle, KYC status, payout method, SOS protocol, support	✅
`notifications`	`/(tabs)/alerts`	New delivery, incentive, document expiry, payment disbursed, order reassigned, tier upgrade	✅
`combined-delivery-v2`	—	Multi-stop sequence	⏭
`rider-ui-kit`	—	Component spec: primary/destructive/disabled buttons, status chips (online/offline/syncing/sync failed), dispatch card, persistent active-job bar	reference

Tab bar (from the design): Tasks · Earnings · Alerts · Profile, plus Home as the start screen. Design system: the customer app's tokens (ink `#050605`, electric lime `#B3F400`), high contrast, 56 pt primary buttons, one primary action per screen.

---

2. Geography: state, city, zone, hub

2.1 Model

```
geo_states (MH) ──< geo_cities (latur, pune) ──< geo_zones (latur-central, latur-east …) ──< hubs (store or start point)
                                  |                                  |
                        ol_service_areas (today)      polygon (PostGIS)
```

- **State** — tax and language defaults, regulatory text.
- **City** — the market (matches `city_id` everywhere: orders, stores, catalogue, admin scopes). `ol_service_areas` keeps its centre and radius as the outer boundary; the city gets a real polygon when drawn.
- **Zone (locality)** — a polygon of a few square km (a ward or a cluster of localities). Dispatch, surge and rider supply are managed per zone. A rider's operating zone and its neighbours are where they receive offers.
- **Hub** — a point: a linked store, a dark store, or a rider's chosen start point (design: *Primary Start Hub*). Store riders are anchored to their store's hub.

2.2 How every record is scoped

Record	state	city	zone	how it's set
Rider	✓	✓	✓ operating zone (+ neighbours)	rider picks zone and hub; server validates the pin is inside the zone
Store / merchant	✓	✓	✓	from the store's map pin
Order / delivery job	✓	✓	pickup zone and drop zone	server computes from coordinates, never from the phone
Admin (Master)	—	✓ scope	optional	`platform_admin_scopes`
Pricing, surge, earnings rules	—	✓	optional override	Master configuration (D12 tiers)

2.3 Zone lookup

`select id from geo_zones where city_id = $1 and st_contains(boundary, st_point($lng, $lat))` — with a GiST index. Points outside every zone are "out of service" (the customer app already refuses them by radius).

---

3. GPS alignment and navigation (Swiggy-grade)

The aim: the rider reaches the right door on the first attempt, and every mile is accounted for.

3.1 Where coordinates come from

Point	Source	Stored with
Store pickup	store's pin, set once by the merchant and confirmed by the first rider arrival	`lat`, `lng`, `accuracy_m`, `source` (`merchant_pin` / `rider_confirmed`), entrance note, photo
Customer drop	customer's map pin in the address picker (already built) + flat/floor/landmark text	`lat`, `lng`, `accuracy_m`, `source` (`customer_pin` / `geocoded` / `rider_corrected`), `confidence`
Rider	phone GPS (fused provider)	`lat`, `lng`, `accuracy_m`, `speed`, `heading`, `recorded_at`, battery

Address confidence (0–1) rises when a rider delivers within 50 m of the pin and falls when a *Wrong address* dispute is raised. Low-confidence addresses show the rider a warning and the customer is asked to re-pin on their next order.

3.2 Arrival geofences

Event	Rule
"Arrived at pickup" becomes available	rider within **150 m** of the store pin (accuracy ≤ 50 m)
Auto-prompt "You have arrived"	within **100 m** of the drop pin for 10 s
Arrived pressed far away	beyond **500 m** → allowed with a reason, flagged for review (no silent block: GPS can be poor between buildings)
Delivery proof location	proof records the rider's coordinate; > 300 m from the drop pin → the job is flagged

3.3 Navigation handoff

The app opens the navigation app on **coordinates**:

- Android: `google.navigation:q=<lat>,<lng>&mode=l` (two-wheeler mode where available), fallback `mode=d`.
- iOS: `comgooglemaps://?daddr=<lat>,<lng>&directionsmode=driving`, fallback Apple Maps `maps://?daddr=<lat>,<lng>`.
- Web fallback: `https://www.google.com/maps/dir/?api=1&destination=<lat>,<lng>&travelmode=driving`.

The in-app map (react-native-maps, Google provider) shows rider, store and drop pins, the route line and the entrance note; it is for orientation, turn-by-turn stays in Google Maps.

3.4 Tracking cadence (R8)

Rider state	Location updates	Transport
Offline	none	—
Online, no job	heartbeat every 30 s (coarse, ≤ 100 m)	`POST /api/rider/heartbeat`
On a job, moving	every 5 s or 25 m, whichever first	batched `POST /api/rider/jobs/:id/track` every 10 s; Android foreground service with a visible notification
On a job, stationary	every 30 s	same
No network	queued on the phone, sent in order when back (design: *syncing queued tasks*)	same endpoint, `recorded_at` preserved

The customer's tracking screen gets the rider position through a Realtime broadcast on `order:<order_id>` (D5: phones listen only), throttled to one update every 5 s.

3.5 ETA

V1: Google Routes API (server-side, cached 60 s per job leg) with the two-wheeler mode where supported; fallback straight-line × 1.35 at 22 km/h. Used for dispatch ranking, the offer card and the customer's ETA.

---

4. Delivery engine

4.1 Job states (from the design's *OneLocal Dispatch State Engine*)

#	State	Meaning	Exception branch (design)	Exception state
01	`available` *(rider state)*	Online and tracking	Offline break	rider `offline`
02	`offered`	Offer shown, timer running	Auto-reject / timeout	→ re-offer
03	`accepted`	Rider assigned, route locked	Re-assign requisition	→ `offered` to next rider
04	`to_pickup`	Travelling to merchant	Vehicle breakdown	`exception:vehicle` → re-assign
05	`at_pickup`	Waiting for dispatch	Order not ready	`exception:not_ready` (wait timer, compensation)

06	`pickup_verified`	Items / barcode checked	Mismatch / incomplete order	`exception:mismatch` / `exception:incomplete`
07	`picked_up`	Package secured on board	Safety incident alert	`exception:safety` (SOS)
08	`to_drop`	Travelling to customer	Flat tyre / accident	`exception:vehicle` / `exception:safety`
09	`at_drop`	At customer location	Customer unavailable / wrong address	`exception:unavailable` / `exception:address`
10	`handover`	Handing over	Rejected by customer	`exception:refused` → return to store
11	`proof`	OTP / photo / signature	Failed verification	`exception:proof_failed` (5 wrong OTPs)
12	`delivered`	Funds / cash recorded	—	—

Terminal states: `delivered`, `failed` (after operations resolves an exception), `returned`, `cancelled`. Steps only move forward; every change carries the expected `version` so two taps can't double-advance.

4.2 Dispatch (R6)

1. Merchant accepts the order (VINII POS / Partner) → job `created` with pickup = store hub, drop = customer pin.
2. **Candidates:** online riders, status active, not on a job (or finishing one within 5 min, V2), cash in hand below limit, vehicle allowed for the category, within the pickup zone or its neighbours.
3. **Order:** riders linked to that store first; then Solo riders by road ETA to pickup; ties → fewer jobs today (fairness).
4. **Auto-Accept riders** are assigned immediately (design: *Job auto-accepted*); **Manual** riders get a 30 s offer (design: *job-offer-manual*); on decline or timeout the next candidate.
5. After 3 rounds with no rider → the merchant and operations are alerted; the job stays open with a widening radius (+2 km per round, max 8 km).
6. **Ready-time sync:** the rider is dispatched so they arrive when the merchant's prep time ends (prep minutes from `ol_merchant_channels`).

4.3 Money on a job

- Earnings = base + distance (pickup→drop km × rate) + peak/surge + wait compensation + tip (100 % to rider), from the city's active rule (`rule_version` stored on the job). Samples from the design: base ₹45, distance ₹25, peak ₹15, tip ₹10 → ₹95.
- Cash on delivery: collected amount → `rider_cash_ledger` (never edited); going online is blocked above the city's cash limit until deposited.
- Offer card shows the estimated payout; the final payout is fixed at delivery.

---

5. API contracts

All routes live on the One Local server. Base: `https://<one-local-domain>/api`. JSON in and out.

Headers: `authorization: Bearer <supabase access token>` (rider signed in by phone OTP) · `x-app: onlatur-rider` · `x-app-version` · `x-runtime` · `x-platform` · `idempotency-key` on every POST that changes state. **Errors:** `{ "detail": "human message", "code": "machine_code" }` with HTTP status. Codes used below: `not_enrolled`, `suspended`, `cash_limit`, `offer_expired`, `already_taken`, `version_conflict`, `too_far`, `otp_invalid`, `otp_locked`, `out_of_zone`. **Who may call:** every `/rider/*` route requires an active `riders` row for the caller (D2), except onboarding routes which require only a signed-in user.

5.1 Onboarding

Method	Route	Body → Response
POST	`/rider/register`	`{ full_name, referral_code? }` → `{ rider_id, status: "draft" }` (phone comes from the verified login)
PUT	`/rider/profile`	`{ full_name, photo_asset_id, emergency_phone, language }` → rider
GET	`/geo/zones?lat=&lng=`	→ `{ state, city, zone, neighbours[] }` or `out_of_zone`
GET	`/geo/cities/:city/zones`	→ zones with name and polygon (simplified) for the picker map
PUT	`/rider/hub`	`{ zone_id, lat, lng, address, landmark }` → hub (validated inside zone)
PUT	`/rider/type`	`{ type: "store" | "solo" }` (`taxi` → 409 coming soon)
POST	`/rider/store-link`	`{ invite_code }` → `{ organization, store, manager_name, daily_pay_text }`
POST	`/rider/kyc/digilocker/start`	→ `{ redirect_url }` (DigiLocker consent)
POST	`/rider/kyc/documents`	`{ kind: "aadhaar"|"pan"|"dl"|"rc"|"selfie", asset_id, number? }` → document with status
PUT	`/rider/vehicle`	`{ class: "2w"|"3w"|"4w", ownership: "own"|"rent", registration_no, rc_asset_id }`
PUT	`/rider/preferences`	`{ categories: ["quick_drop","on_order","pick_drop"], acceptance: "auto"|"manual", multi_store: bool, store_ids[] }`
PUT	`/rider/payout`	`{ method: "bank", holder, account_no, ifsc }` or `{ method: "upi", vpa }` → `{ verified_name, status }` (penny-drop / VPA lookup)
POST	`/rider/submit`	→ `{ status: "submitted" }`
GET	`/rider/status`	→ `{ status: draft|submitted|verification_pending|approved|rejected|documents_expired|suspended, reason?, expired_documents[], eta_text }`
POST	`/rider/uploads`	`{ kind, content_type }` → `{ asset_id, upload_url }` (signed upload to Supabase Storage, private bucket)

5.2 Working day

Method	Route	Body → Response
GET	`/rider/me`	→ profile, status, zone, hub, acceptance mode, cash in hand, today `{ earnings, jobs }`, yesterday
POST	`/rider/availability`	`{ online: bool, lat, lng, accuracy_m }` → `{ online }` · errors `cash_limit`, `out_of_zone`, `suspended`
POST	`/rider/heartbeat`	`{ lat, lng, accuracy_m, battery, app_state }` → `{ ok, server_time }`
PUT	`/rider/push-token`	`{ expo_push_token, platform }` → ok

GET	`/rider/offers/current`	→ `{ offer_id, job, payout_estimate, surge_multiplier, pickup{name,area,distance_km}, drop{area,distance_km}, eta_min, note, expires_at }` or `204`
POST	`/rider/offers/:id/accept`	→ job · errors `offer_expired`, `already_taken`
POST	`/rider/offers/:id/decline`	`{ reason }` → ok
GET	`/rider/jobs/current`	→ job (see 5.4) or `204`
GET	`/rider/jobs/:id`	→ job (own jobs only)
POST	`/rider/jobs/:id/step`	`{ to: "to_pickup"|"at_pickup"|"to_drop"|"at_drop"|"handover", version, lat, lng, accuracy_m, reason? }` → job · errors `version_conflict`, `too_far` (soft: needs `reason`)
POST	`/rider/jobs/:id/pickup/verify`	`{ method: "scan"|"code"|"bypass", code?, items: [{ id, ok }], reason? }` → `{ result: "verified"|"mismatch"|"incomplete", expected_sku?, missing[] }`
POST	`/rider/jobs/:id/track`	`{ points: [{ lat, lng, accuracy_m, speed, heading, recorded_at }] }` (≤ 60 points) → ok
POST	`/rider/jobs/:id/proof`	`{ method: "otp", code }` · `{ method: "photo", asset_id }` · `{ method: "signature", asset_id }` + `{ lat, lng, cash_collected? }` → job `delivered` with earnings · errors `otp_invalid` (attempts left), `otp_locked`
POST	`/rider/jobs/:id/exception`	`{ kind: "not_ready"|"mismatch"|"incomplete"|"vehicle"|"safety"|"unavailable"|"address"|"refused"|"other", note?, asset_id?, lat, lng, corrected_lat?, corrected_lng? }` → exception with next allowed actions
POST	`/rider/sos`	`{ job_id?, lat, lng }` → ok (alerts operations, shares live location)
POST	`/rider/jobs/:id/call`	→ `{ number }` (V1: customer's number during the job only; V2: masked number)

5.3 Money and history

Method	Route	Response
GET	`/rider/earnings?range=today|week|month`	totals, per-day series, split (order pay, bonus, tips), insights
GET	`/rider/earnings/jobs/:id`	route and payout calculation lines
GET	`/rider/jobs?cursor=`	task history (delivered / failed)
GET	`/rider/statements/:week.pdf`	weekly statement PDF
GET	`/rider/cash`	cash in hand, ledger lines, deposit instructions
GET	`/rider/notifications?cursor=`	alerts list; `POST /rider/notifications/read`

5.4 Job object (what the rider sees)

```
{
  "id": "job_7d1c…", "order_ref": "#9824", "state": "to_drop", "version": 7,
  "category": "on_order", "city_id": "latur", "pickup_zone": "latur-central", "drop_zone": "latur-east",
  "pickup": { "name": "Gourmet Kitchen", "address": "Sector 4, Block B", "lat": 18.4088, "lng": 76.5604,
              "accuracy_m": 8, "entrance_note": "Side gate", "phone_masked": true },
  "drop":   { "area": "Shanti Enclave", "address": "Apt 4B, Shanti Enclave, Sector 15", "lat": 18.4211, "lng": 76.5793,
              "accuracy_m": 12, "source": "customer_pin", "confidence": 0.86, "landmark": "Opp. temple",
              "customer_first_name": "Amit", "instructions": "Leave at door, contactless" },
  "items_count": 3, "cash_to_collect": 0, "proof_methods": ["otp", "photo"],
  "payout_estimate": 85.5, "eta": { "to_pickup_min": 6, "to_drop_min": 14 }, "distance_km": 4.2,
  "deadlines": { "pickup_by": "2026-09-26T13:10:00+05:30", "drop_by": "2026-09-26T13:30:00+05:30" }
}
```

Privacy: the full drop address appears only after `pickup_verified`; the customer's phone only during the job.

5.5 Connections to the other apps

From → to	Route / channel	Purpose
Customer app → server	`GET /api/orders/:id` (exists) + Realtime `order:<id>`	Order state, rider first name, live position, ETA, delivery OTP
Merchant (Partner app) → server	`POST /api/partner/orders/:id/accept` (exists) → creates the delivery job; `…/ready` (exists) → notifies rider	Dispatch start, ready signal
Server → rider	Expo push (offer, reassignment) + Realtime `rider:<rider_id>`	Offers and updates
Master → server	`/api/admin/dispatch/*` (to build): zones, hubs, riders approve/reject/suspend, live map, reassign, resolve exceptions, earnings rules, cash deposits	Operations

6. Schema (proposed migrations in Repo-1-, D7; additive)

Owner: **OnLatur** (delivery), except `geo_*` which belong to the Master (shared geography). All tables: RLS on, no public policies, server-only writes, every column commented.

```
create extension if not exists postgis;

-- Geography (Master-owned)
create table geo_states (code text primary key, name text not null, default_language text);
create table geo_cities (id text primary key references ol_service_areas(city_id), state_code text not null references geo_states(code),
  name text not null, boundary geography(multipolygon), timezone text not null default 'Asia/Kolkata');
create table geo_zones (id text primary key, city_id text not null references geo_cities(id), name text not null,
  boundary geography(polygon) not null, neighbours text[] not null default '{}', active boolean not null default true);
create index geo_zones_boundary on geo_zones using gist (boundary);
create table hubs (id uuid primary key default gen_random_uuid(), zone_id text not null references geo_zones(id),
  kind text not null check (kind in ('store','dark_store','rider_start')), organization_id uuid references organizations(id),
  store_id text references ol_stores(id), name text, location geography(point) not null, address text, landmark text,
  accuracy_m int, source text, entrance_note text, photo_asset_id uuid);

-- Riders
create table riders (id uuid primary key default gen_random_uuid(), user_id uuid not null unique references auth.users(id),
  phone text not null, full_name text not null, photo_asset_id uuid, emergency_phone text, language text default 'en',
  type text not null check (type in ('store','solo','taxi')), status text not null default 'draft' check (status in
  ('draft','submitted','verification_pending','approved','rejected','documents_expired','suspended')), status_reason text,
  city_id text references geo_cities(id), zone_id text references geo_zones(id), hub_id uuid references hubs(id),
  organization_id uuid references organizations(id), acceptance text not null default 'manual' check (acceptance in ('auto','manual')),
  categories text[] not null default '{on_order}', multi_store boolean not null default false,
  cash_limit numeric(10,2), referral_code text, referred_by uuid references riders(id),
  created_at timestamptz not null default now(), updated_at timestamptz not null default now());
create table rider_store_links (rider_id uuid references riders(id), store_id text references ol_stores(id),
  priority int not null default 1, status text not null default 'active', primary key (rider_id, store_id));
create table rider_documents (id uuid primary key default gen_random_uuid(), rider_id uuid not null references riders(id),
  kind text not null check (kind in ('aadhaar','pan','dl','rc','selfie','permit','fitness')), number_masked text,
  asset_id uuid, source text check (source in ('digilocker','upload')), status text not null default 'pending'
  check (status in ('pending','verified','rejected','expired')), expires_on date, reviewed_by uuid, reviewed_at timestamptz);
create table rider_vehicles (rider_id uuid primary key references riders(id), class text not null check (class in ('2w','3w','4w')),
  ownership text not null check (ownership in ('own','rent')), registration_no text, rc_document_id uuid references rider_documents(id));
create table rider_payout_methods (id uuid primary key default gen_random_uuid(), rider_id uuid not null references riders(id),
  method text not null check (method in ('bank','upi')), holder_name text, account_last4 text, account_token text, ifsc text,
  vpa text, verified_name text, verified_at timestamptz, is_primary boolean not null default true);
  -- full account numbers are stored encrypted (pgsodium / Vault), never in plain columns
create table rider_presence (rider_id uuid primary key references riders(id), online boolean not null default false,
  location geography(point), accuracy_m int, battery int, app_state text, last_seen_at timestamptz,
  current_job_id uuid);
create index rider_presence_location on rider_presence using gist (location) where online;

-- Delivery jobs
create table delivery_jobs (id uuid primary key default gen_random_uuid(), ol_order_id uuid references ol_orders(id),
  pos_order_id uuid, organization_id uuid references organizations(id), city_id text not null references geo_cities(id),
  pickup_zone_id text references geo_zones(id), drop_zone_id text references geo_zones(id), category text not null,
  state text not null default 'created', version int not null default 0, rider_id uuid references riders(id),
  pickup jsonb not null, drop jsonb not null, pickup_point geography(point) not null, drop_point geography(point) not null,
  items_count int, cash_to_collect numeric(10,2) not null default 0, distance_km numeric(6,2),
  otp_hash text, otp_attempts int not null default 0, proof_methods text[] not null default '{otp,photo}',
  payout_estimate numeric(10,2), earnings_rule_version int, pickup_by timestamptz, drop_by timestamptz,
  created_at timestamptz not null default now(), updated_at timestamptz not null default now());
create table delivery_offers (id uuid primary key default gen_random_uuid(), job_id uuid not null references delivery_jobs(id),
  rider_id uuid not null references riders(id), round int not null, mode text not null check (mode in ('auto','manual')),
  offered_at timestamptz not null default now(), expires_at timestamptz not null,
  outcome text check (outcome in ('accepted','declined','timed_out','withdrawn')), reason text, answered_at timestamptz);
create unique index one_accepted_offer_per_job on delivery_offers (job_id) where outcome = 'accepted';
create table delivery_events (id bigserial primary key, job_id uuid not null references delivery_jobs(id), from_state text,
  to_state text not null, actor text not null, rider_id uuid, location geography(point), accuracy_m int, reason text,
  app text, created_at timestamptz not null default now());
create table delivery_track_points (job_id uuid not null references delivery_jobs(id), recorded_at timestamptz not null,
  location geography(point) not null, accuracy_m int, speed real, heading real, primary key (job_id, recorded_at));
  -- kept 30 days, then summarised into distance travelled
create table delivery_pickup_checks (job_id uuid primary key references delivery_jobs(id), method text not null,
  code_scanned text, code_expected text, result text not null, missing jsonb, checked_at timestamptz not null default now());
create table delivery_proofs (id uuid primary key default gen_random_uuid(), job_id uuid not null references delivery_jobs(id),
  method text not null check (method in ('otp','photo','signature')), asset_id uuid, location geography(point),
  distance_from_drop_m int, created_at timestamptz not null default now());
create table delivery_exceptions (id uuid primary key default gen_random_uuid(), job_id uuid not null references delivery_jobs(id),
  rider_id uuid references riders(id), kind text not null, note text, asset_id uuid, location geography(point),
  corrected_point geography(point), status text not null default 'open' check (status in ('open','resolved')),
  resolution text, resolved_by uuid, resolved_at timestamptz, created_at timestamptz not null default now());
create table address_quality (address_id uuid primary key, confidence numeric(3,2) not null default 0.7,
  deliveries int not null default 0, disputes int not null default 0, last_rider_point geography(point), updated_at timestamptz);

-- Money
create table earnings_rules (id serial primary key, city_id text not null references geo_cities(id), version int not null,
  base numeric(8,2) not null, per_km numeric(8,2) not null, peak jsonb not null default '[]', wait_free_min int not null default 10,
  wait_per_min numeric(6,2) not null default 0, active_from timestamptz not null, unique (city_id, version));
create table rider_earnings (job_id uuid primary key references delivery_jobs(id), rider_id uuid not null references riders(id),
  base numeric(8,2), distance numeric(8,2), peak numeric(8,2), wait numeric(8,2), tip numeric(8,2), bonus numeric(8,2),
  total numeric(10,2) not null, rule_version int not null, created_at timestamptz not null default now());
create table rider_cash_ledger (id bigserial primary key, rider_id uuid not null references riders(id), job_id uuid,
  kind text not null check (kind in ('collected','deposited','adjustment')), amount numeric(10,2) not null,
  recorded_by uuid, created_at timestamptz not null default now());   -- insert-only (no update/delete grants)
create table rider_payouts (id uuid primary key default gen_random_uuid(), rider_id uuid not null references riders(id),
  period_start date, period_end date, amount numeric(10,2) not null, method_id uuid references rider_payout_methods(id),
  status text not null default 'pending' check (status in ('pending','paid','failed')), reference text, paid_at timestamptz);
create table rider_notifications (id bigserial primary key, rider_id uuid not null references riders(id), kind text not null,
  title text not null, body text, data jsonb, read_at timestamptz, created_at timestamptz not null default now());
```

Push tokens use `app_devices` (Master, D2). Rider identity is `auth.users` by phone (D1); the same person can also be a One Local shopper.

---

7. End-to-end flows

7.1 Onboarding (Store Rider)

```
Splash → Intro (3) → Location permission → Sign in (phone) —new→ Register → OTP → Profile
  → Zone (map, detected) → Start hub → Confirm pin → Notifications permission → Rider type: Store
  → Link to store (invite code) → KYC (DigiLocker + selfie + DL) → Vehicle (+RC) → Categories → Stores
  → Acceptance mode → Priority rules → Submit → Approval pending —→ Approved → Home (offline)
                                                  └→ Rejected / documents expired → re-upload
```

Solo Rider: same, without store link and priority rules, with bank or UPI payout before submit.

7.2 A delivery

```
Customer places order —→ Merchant accepts (Partner app) —→ job created (pickup = store hub, drop = customer pin)
  —→ dispatch: linked riders → nearby Solo (ETA) —→ Auto: assigned | Manual: 30 s offer → accept
  —→ to_pickup (navigate on coordinates) —→ at_pickup (within 150 m) —→ verify: scan QR / code / checklist
       └ not ready → wait timer (compensation after 10 min) · mismatch / incomplete → report → merchant fixes
  —→ picked_up —→ to_drop (tracking every 5 s; customer sees live map) —→ at_drop (100 m auto-prompt)
       └ unavailable → mandatory wait (call, chat) → escalate · wrong address → share location / dispute
  —→ handover —→ proof: OTP (default) / photo / signature —→ delivered: earnings + cash ledger; customer notified
```

7.3 Exceptions (who resolves)

Exception	Rider action	Resolved by	Outcome
Not ready	wait, report delay	merchant	continues; wait pay after threshold
Mismatch / incomplete	report	merchant	corrected → continue, or cancel with merchant fault
Vehicle breakdown	report	server	re-dispatch to another rider (picked-up packages: operations arranges transfer)
Customer unavailable	wait timer, call ×2	operations	return to store or leave with proof (policy per category)
Wrong address	share location, dispute	operations / customer	corrected pin → continue; address confidence lowered
Refused	record reason	operations	return to store
Safety / SOS	trigger	operations	job paused; emergency contacts; incident record
Proof failed (5 OTP tries)	photo fallback	operations	approve photo proof or return

8. Non-functional

- **Alert reliability:** offers by push (high-priority Android channel with custom sound; iOS time-sensitive) and Realtime; offer card within 3 s in foreground, 10 s in background (PRD M0 checks).
- **Offline:** every state change is queued with its `idempotency-key` and original `recorded_at`; the connectivity banner follows the design's four states.
- **Battery:** location sampling per §3.4; no tracking when offline.
- **Privacy:** customer address after pickup verification only; phone during the job only; track points kept 30 days; KYC images in a private bucket with signed, short-lived URLs; account numbers encrypted.
- **Security:** only riders with `status = approved` can go online; every route checks the rider owns the job; server decides state, prices and geofence results.
- **Store policy:** foreground-service location during jobs only (Play Store "location in use during active delivery" declaration); iOS "when in use" plus background mode only during a job.

9. Build milestones

Milestone	Delivers	Done when
M0 — It rings	Rider app shell (sign-in by phone with test numbers, Home online/offline, Alerts), push token, test offer card with sound and countdown	the four M0 checks in the earlier PRD pass on a real Android phone
M1 — Onboarding	Screens 1–30 (Store and Solo), geography tables and zone picker, uploads, status screens, Master approve/reject	a test rider goes from install to *You're approved* with a Master approval
M2 — One job, simulated	Delivery tables, dispatch (linked-first, ETA, timeout, re-offer), all job screens, OTP/photo/signature, earnings, cash ledger	a server-created job completes; two riders racing for one offer → one winner
M3 — Real order	Customer order → Partner accept → job → rider → customer live tracking with OTP	one real Latur order delivered with every step visible to the customer
M4 — Exceptions and money	All exception flows, SOS, weekly statements, cash deposit, payout run (manual), documents-expiry job	a week of pilot deliveries with the cash ledger balancing

Needed from the founder when we reach them: SMS provider account (MSG91) and DLT registration (M1 for real numbers); Google Maps Platform key with Routes API (M2); Firebase project for Android push and an Expo account (M0); DigiLocker partner onboarding (M1 — until then KYC is upload + manual review); a payout provider decision (V2); approval of the Repo-1- migrations.

10. Test plan (end to end)

Area	Cases
Sign-in	new vs returning phone; wrong OTP; resend timer; same phone already a shopper (one account, D1)
Onboarding	zone detected vs chosen; hub pin outside zone refused; invite code invalid; KYC rejected → re-upload; document expiry blocks going online
Availability	cash above limit blocks online; out-of-zone blocks online; heartbeat silence 60 s → treated offline
Dispatch	linked rider preferred; manual timeout → next rider; auto-accept; two riders accept same offer → one wins; 3 rounds → merchant alerted
Geofence	arrived at pickup 120 m ✓, 600 m → reason required; proof 350 m from pin → flagged
Navigation	Google Maps opens on coordinates (Android, iOS, no Google Maps installed → Apple Maps / web)
Job	steps can't be skipped or repeated; app killed mid-job resumes at the right screen; offline step queued and replayed in order
Proof	OTP right; wrong ×5 → locked → photo; signature for Pick & Drop
Exceptions	every row in §7.3 end to end, including customer-side messages
Money	earnings match the rule version; tip 100 %; cash ledger insert-only; weekly statement totals
Privacy	address hidden before pickup verification; phone hidden after delivery; another rider's job → 404